

One-Pager System: Options, Architecture & Design

Comprehensive guide to creation options, architecture, and design possibilities

One-Pager System: Options, Architecture & Design

Created: 2026-01-11

Philosophy

> "Physical constellation of crystallized knowledge inside spacetime through the refraction of light"

Creates perfect 2-page (front/back) printable documents from any content.

OPTIONS: How to Create

1. Direct OnePager Class

```
from waft import OnePager
OnePager.from_markdown("# Title\n\nContent", title="Doc").generate()
```

Simple, direct, content-aware.

2. DocumentBuilder Integration

```
from waft import DocumentBuilder
DocumentBuilder.field_guide(title="Doc", content="...", exact_pages=2).save("out.pdf")
```

Unified API, constraint-aware.

3. Global Cursor Command

```
/one-pager file:README.md title:"README"
```

Quick access, no code.

4. CLI Script

```
python3 scripts/create_one_pager.py file:README.md
```

Scriptable, batch processing.

5. Chat-Based

```
waft-one-pager-chat
```

Automatic from chat sessions.

ARCHITECTURE

Components

OnePager: Content processing, HTML generation, style rotation, PDF generation

Template: Jinja2 template, CSS styling, printer-friendly

DocumentBuilder: Constraint-aware, feedback loop, CSS adjustment

Flow

```
Content → Processing → Style → Template → PDF → Output
```

Style Rotation

Sections: story-section, boxed-section, highlight-section, minimal-section

Headers: ", boxed, highlight, underlined

Lists: ", custom-bullets, checkmarks, dashed, boxed

Paragraphs: ", indented, highlight, compact

Code: ", boxed, minimal

HOW IT WORKS

Pipeline

1. **Input Detection:** Auto-detects type (markdown, text, code, dict, JSON)
2. **Format Conversion:** Converts to HTML with structure preservation
3. **Style Application:** Rotates through style variants
4. **Template Rendering:** Combines content with template
5. **PDF Generation:** Renders HTML to PDF (WeasyPrint)
6. **Output:** Saves to `workefforts/onepagers/[title][date].pdf`

Content Handling

Long Content: Preserves headers, keeps first paragraph per header, preserves code blocks, truncates redundant paragraphs.

Short Content: Adds summary sections, expands with metadata, ensures minimum 2-page output.

Constraints: Simple (direct) or Advanced (feedback loop adjusts CSS until 2 pages).

DESIGN POSSIBILITIES

1. **Simple Template** (Current) - Single template, style rotation
2. **Template Variants** - Multiple base templates
3. **Evolutionary Templates** - Templates evolve based on usage patterns
4. **Content-Aware Design** - Template adapts to content type
5. **Constraint-Aware Design** - Active feedback loop, guaranteed 2-page

6. Study Gym Integration - Scientific method analysis, pattern learning

7. Hybrid Approach - Combine multiple designs

FUTURE POSSIBILITIES

Genome-Based Evolution

Each one-pager has a "genome" (style composition, metrics). Registry tracks genomes. Pattern analysis identifies successful combinations. Templates evolve.

Full Evolutionary System

SPAWN (variants), MUTATE (hot-swap), GYM_EVAL (fitness), DEATH/SURVIVAL (selection), Conjugate (hybrid from two parents).

Selection Mechanisms

Fitness-proportional, Tournament, Elitism, Diversity maintenance.

Mutation Types

Point mutation, Crossover, Deletion/Insertion, Inversion.

User Feedback

Ratings, Notes, Weight analysis, Prioritize highly-rated patterns.

Multi-Format

PDF (current), HTML, Markdown, LaTeX.

Batch Processing

Multiple files, Collections, Auto-binder, Bulk style application.

RECOMMENDED APPROACH

Quick Use: Option 1 or 3

Production: Option 7 (Hybrid)

Research: Option 6 (Study Gym)

Status

Current: Simple template with style rotation

Planned: Genome-based evolution, pattern analysis

Future: Full evolutionary system, user feedback, multi-format

Living, evolving tool that learns from each generation.